A Dedicated Edge System and MCGI-YOLO for Fabric Defect Detection and Closed-Loop Control in Circular Knitting Machines

Kangzhao Tan[#], Jiarui Zhang[#], Junjie Yu, Heng Wu, Jing Lin[*], and Chunhua He[*]

***Abstract*—Defect detection is essential for circular knitting production, where persistent needle or yarn abnormalities lead to repeated fabric defects, resulting in severe material waste and economic losses. However, existing studies are generally not tailored to circular knitting machines and tend to be cost-prohibitive, which hinders their large-scale industrial deployment. To address this issue, we first design and implement a low-cost, dedicated defect detection and closed-loop control system for circular knitting machines. The system consists of a global-shutter camera, a Hall sensor, and a self-developed edge device. The Hall sensor signal triggers image acquisition and defect detection. When a defect is detected, the device stops the circular knitting machine to interrupt defect propagation and alerts the operator for corrective intervention. To further improve detection accuracy for diverse fabric defect morphologies, we propose a lightweight detector termed MCGI-YOLO. To model the directional continuity of horizontal and vertical elongated defects, morphology-aware context encoding (MCE) augments backbone features with spatially extended directional context. To accommodate variations in defect morphology and scale, gated interpolation upsampling (GIU) adapts the reconstruction of higher-level context before fusion with local features, using a compact content-dependent gate. On the test set of a self-collected dataset from a real circular-knitting production line, MCGI-YOLO achieves 97.2% mAP50 and 65.5% mAP50–95, improving on the YOLOv8n baseline by 6.7 and 7.1 percentage points, respectively, with a detection throughput of 22 FPS when deployed on the RK3566 edge device. These results demonstrate that the proposed system offers a cost-effective solution for large-scale deployment in real-time defect detection for circular-knitting production.**

***Index Terms*—Circular knitting machine, fabric defect detection, edge computing, MCE(morphology-aware context encoding), GIU(gated interpolation upsampling), YOLO.**

I. Introduction

CIRCULAR knitting machines are are key production equipment for high-volume weft-knit fabric, which is widely used in everyday clothing such as T-shirts and sportswear [1]–[3]. The International Textile Manufacturers Federation reported approximately 24,500 global shipments of circular knitting machines in 2025 [4]. Globally, approximately 92 million tonnes of textile waste are generated annually, highlighting the importance of reducing avoidable material losses [5]. Conventional quality checks are often performed after knitting is completed. A persistent needle or yarn abnormality can therefore generate repeated defects along a fabric roll before discovery, accumulating substantial fabric waste and economic losses [6]–[8]. To prevent such losses, real-time detection during knitting is needed to identify these defects early and enable timely intervention before further defective fabric is produced.

Edge computing offers a practical pathway toward such real-time detection, and has been increasingly adopted for industrial defect detection. In textile applications, Song et al. deployed EfficientDet-D0 on an NVIDIA Jetson TX2 and evaluated five fabric datasets covering dark red, grid, camouflage, digitally printed, and light blue fabrics [9]. Machado et al. deployed YOLO11n on a Jetson Orin Nano to detect solid-color cotton and linen fabrics on a fabric rolling line [10]. Yan et al. deployed an improved YOLOv5s model on a Huawei Ascend 310-based Atlas platform to detect warp-knitted lace on warp-knitting machines [11].

Beyond textile detection, Wang et al. deployed an improved YOLOv5s model on a Jetson Xavier NX for rail surface defect detection [12]. Li et al. deployed LiteYOLO-ID on a Jetson TX2 NX for insulator defect detection [13], and Zhao et al. implemented KAM-Net on a Xilinx ZCU102 FPGA for glass defect segmentation [14]. These studies predominantly adopt general-purpose edge-computing platforms or FPGA development boards. However, such platforms are not tailored to the sensing, control, and installation constraints of circular knitting machines. Their relatively high hardware cost, together with the additional integration effort required for image acquisition and machine control, hinders economically viable deployment across large numbers of machines.

In terms of detection algorithms, YOLO-based detectors have been widely explored for industrial defect detection because of their efficient inference and explicit defect localization [15]. Yang et al. developed YOLOV4-TinyS using lightweight convolutions, shallow-feature fusion, and attention, and evaluated

Manuscript received October 12, 2026; revised xxxx xx, 2026; accepted xxxx xx, 2026. ([#]These authors contributed equally: Kangzhao Tan, Jiarui Zhang) ([*]Corresponding authors: Jing Lin and Chunhua He).

Kangzhao Tan, Jiarui Zhang, Junjie Yu, and Chunhua He are with the School of Computer, Guangdong University of Technology, Guangzhou 510006, China (e-mail: tankangzhao@mails.gdut.edu.cn; zhangjiarui1@mails.gdut.edu.cn; yujunjie2@mails.gdut.edu.cn; hechunhua@pku.edu.cn). Heng Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China (e-mail: heng.wu@foxmail.com). Jing Lin is with the Classroom Management Center, Guangdong University of Technology, Guangzhou 510006, China (e-mail: lj@gdut.edu.cn).

the detector in a conveyor-based fabric detection system [16]. Kang and Li proposed AYOLOv7-tiny for solid-color circular weft fabrics, combining space-to-depth downsampling with hybrid attention to preserve subtle defect information [17]. Chen et al. enhanced YOLOv8 with Swin Transformer blocks, a bidirectional feature pyramid, attention, and a modified localization loss [18]. FA-YOLO combines receptive-field enhancement, attention, and adaptive spatial feature fusion [19]. DGHR-YOLO integrates deformable convolution for morphology adaptation with a high-level screening feature pyramid and lightweight feature processing [20]. Together, these methods advance the representation of subtle defects, irregular shapes, and scale variation. Related work on photovoltaic-cell and pipeline-weld detection also explores task-specific feature pyramids, contextual relations, and lightweight rotated representations [21]–[23]. However, these methods are tailored to their respective detection targets and may not be directly applicable to defect detection on circular knitting machines. Defects on circular knitting machines exhibit diverse morphologies and scales, ranging from horizontal and vertical elongated patterns to localized holes. In addition, modules such as deformable convolution and attention often introduce substantial computational overhead, making them less suitable for deployment on resource-constrained edge devices.

Motivated by these limitations, we design and implement a low-cost, dedicated defect detection and closed-loop control system for circular knitting machines. The system integrates a power adapter, a global-shutter camera with LED illumination, a coaxial serializer/deserializer (SerDes) image-transmission link, a Hall sensor, and a self-developed edge device. The Hall sensor detects machine rotation to trigger image acquisition and defect detection. When a defect is detected, the edge device stops the circular knitting machine, uploads the defect image and record, and pushes an alert to the operator through the cloud platform for corrective intervention.

For defect detection within this system, we develop MCGI-YOLO, a lightweight detector based on YOLOv8n that incorporates the morphology priors for horizontal and vertical elongated defects. MCE supplies spatially extended morphology-aware context while preserving the original local C2f feature paths. GIU addresses the differing reconstruction needs of elongated defects and holes of varying sizes by adaptively adjusting the interpolation of higher-level context before fusion with local lateral features. Together, the two components connect morphology-aware context encoding with spatial reconstruction within a compact parameter budget.

The main contributions of our work are as follows.

(1) We design and implement a dedicated defect detection and closed-loop system for circular knitting machines, integrating image acquisition, camera transmission, edge inference, and local machine-stop control, and demonstrate its deployment on a real production line.

(2) We develop MCE to explicitly incorporate the morphology priors of horizontal and vertical elongated defects. It embeds spatially extended, morphology-aware context in a partial C2f branch while retaining the original local feature extraction paths with small additional overhead.

(3) We design GIU to accommodate the diverse morphologies and scales of circular-knitting defects, including elongated defects and holes of varying sizes. A compact spatial gate adapts a convex combination of two fixed interpolation paths, balancing response replication and neighborhood smoothing in the top-down path before fusion with local lateral features.

The remainder of this paper is organized as follows. Section II presents the defect detection and closed-loop control system. Section III describes MCGI-YOLO and its two components. Section IV defines the dataset and evaluation protocol. Section V reports detector comparisons, component ablation, and model deployment. Section VI concludes the paper.

II. Defect Detection and Closed-Loop Control System

A. System Overview

We design and implement a dedicated defect detection and closed-loop control system for circular knitting machines, as shown in Fig. 1. The system integrates a power adapter, a global-shutter camera with LED illumination, a SerDes image-transmission link, a Hall sensor, a self-developed edge device, and a cloud platform. The closed-loop workflow begins when the operator starts the machine. The Hall sensor detects machine rotation and triggers the edge device to activate the camera and LED illumination for image acquisition and defect detection. When a defect is detected, the edge device immediately sends a stop command to the machine controller to prevent further defect propagation, uploads the defect image and record to the cloud, and pushes an alert notification to the operator. After the operator inspects the reported defect, resolves the production fault, and restarts the machine, the rotation signal reactivates detection. This sequence forms a complete production-–detection-–control loop. The system is organized into four functional subsystems: power supply, image acquisition and transmission, and processor. The following subsections describe their designs and hardware implementation.

B. Power Supply Subsystem

To accommodate different industrial supply voltages across countries and regions, we design a power adapter with a 100–500 V AC input range and a 24 V DC output rated at 3 A. The adapter first supplies this output to the device. The device then supplies the LED illumination. By adjusting the current delivered to the LEDs, the device controls the illumination brightness to suit the image-acquisition conditions. This arrangement integrates LED power delivery and brightness control into the device.

C. Image Acquisition and Transmission Subsystem

The rapid rotation of circular knitting machines requires imaging that preserves defect geometry and fine spatial detail. We therefore select a SmartSens SC130GS global-shutter sensor(SmartSens Technology Co., Ltd., Shanghai, China) at

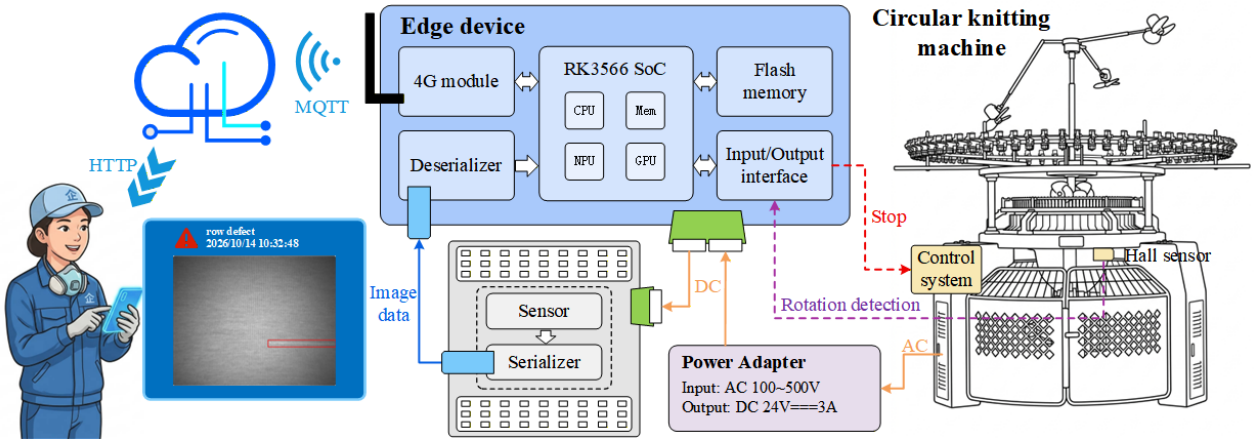


Fig. 1. System overview and subsystem interconnections. The power adapter supplies the device, which powers the LED illumination and controls its brightness by adjusting the output current. Images are transmitted through the serializer, coaxial link, and deserializer to the RK3566 processor. The 4G network connects the device to the cloud for monitoring and defect alerts. Wired input/output interfaces receive the Hall-sensor rotation signal and send stop commands to the machine controller.

1280×1024 resolution, which exposes all pixels simultaneously to avoid motion-induced distortion from sequential row exposure. At a camera-to-fabric distance of approximately 10 cm, the field of view is approximately 16×12 cm, with the 16 cm dimension aligned with the horizontal direction of fabric motion.

Factory machinery creates an electromagnetically noisy environment, and installation requires the camera and edge device to be separated by a cable run. We therefore use serializer/deserializer (SerDes) transmission over shielded coaxial cable to accommodate remote camera placement and the interference environment. Specifically, the THCV241A serializer and THCV242A deserializer(THine Electronics, Inc., Tokyo, Japan) bridge the camera-side and processor-side MIPI CSI-2 interfaces. The devices support a maximum serial line rate of 4.0 Gb/s per lane, corresponding to a rated effective rate of 3.2 Gb/s [24], [25].

The camera is configured to transmit 1280×1024 RAW10 images at 30 frames/s. With $b = 10$ bits per pixel, the active-image payload rate is

$$\begin{aligned} R_{\text{img}} &= WHbf_{\text{cam}} \\ &= 1280 \times 1024 \times 10 \times 30 \\ &= 393.216 \text{ Mb/s}, \end{aligned} \tag{1}$$

where W and H denote the image width and height, and f_{cam} is the camera acquisition rate. This corresponds to 49.152 MB/s, accounting for only 12.3% of the rated single-lane effective capacity, which provides a stable image input for the system's defect detection.

D. Processor Subsystem

The processor is selected according to the detection rate required by fabric motion and the camera field of view. Taking a typical large circular knitting machine as an example, with a machine diameter of 40 in ($D = 1.016$ m) and a rotation period of $T = 2$ s, the fabric travels approximately $\pi D = 3.192$ m per revolution. With an effective field-of-view width of $w = 0.16$ m along the direction of motion, the required frame rate is

$$f_{\min} = \frac{\pi D}{Tw} = \frac{\pi \times 1.016}{2 \times 0.16} \approx 9.97 \text{ frames/s}. \tag{2}$$

We set the effective detection-throughput requirement to 10 frames/s. For this throughput target, we select the Rockchip RK3566(Rockchip Electronics Co., Ltd., Fuzhou, China) to balance processing capability, camera connectivity, and hardware cost. It integrates a quad-core Cortex-A55 CPU, an NPU rated at 1 TOPS for INT8 operations [26].

Table I summarizes the hardware specifications of platforms used in related defect detection studies. While high-performance platforms like the NVIDIA Jetson Orin Nano and Xilinx ZCU102 offer substantial computational power, their costs range from hundreds to over 3,000 USD, which can be prohibitive for large-scale deployment. In contrast, the Rockchip RK3566 provides a cost-effective hardware basis for the proposed system, despite its rated compute capability of only 1 TOPS (INT8). Most notably, at a hardware cost of only 60 USD, our solution offers a significant cost advantage over comparable industrial platforms, making it highly suitable for large-scale deployment in textile factories.

E. Hardware Implementation

Based on the design requirements of the subsystems described above, we develop the corresponding hardware shown in Fig. 2. The camera assembly (A) combines the image sensor and LED illumination. We design a camera-side interface board (B) that integrates the serializer and coaxial connector, and a mainboard (C) built around the RK3566. The mainboard integrates memory and flash storage, the camera deserializer and coaxial connector, a 4G module with SIM-card and antenna interface, input/output interface, and power-management circuitry. This board-level integration brings image reception, local inference, network communication, machine control, and illumination control into a dedicated edge device.

Fig. 3 shows the system installed on a circular knitting machine in a real production line. The highlighted components

TABLE I
HARDWARE PLATFORMS USED IN RELATED DEFECT DETECTION STUDIES.

Platform	Memory (GB)	Compute capability	Price (USD)
NVIDIA Jetson TX2 [9]	8	1.33 TFLOPS (FP16)	599
NVIDIA Jetson TX2 NX [13]	4	1.33 TFLOPS (FP16)	249
NVIDIA Jetson Xavier NX [12]	8	21 TOPS (INT8)	399
NVIDIA Jetson Orin Nano [10]	8	40 TOPS (INT8)	499
Huawei Ascend 310 [11]	8	16 TOPS (INT8)	550
AMD Xilinx ZCU102 [14]	4.5†	3.45 TOPS (INT8)‡	3234
Self-developed Rockchip RK3566 device(Ours)	4	1 TOPS (INT8)	60

†ZCU102 memory is the sum of 4 GB PS DDR4 and 0.5 GB PL DDR4. ‡The 3.45 TOPS value is AMD's reference INT8 peak for three B4096 DPU cores.

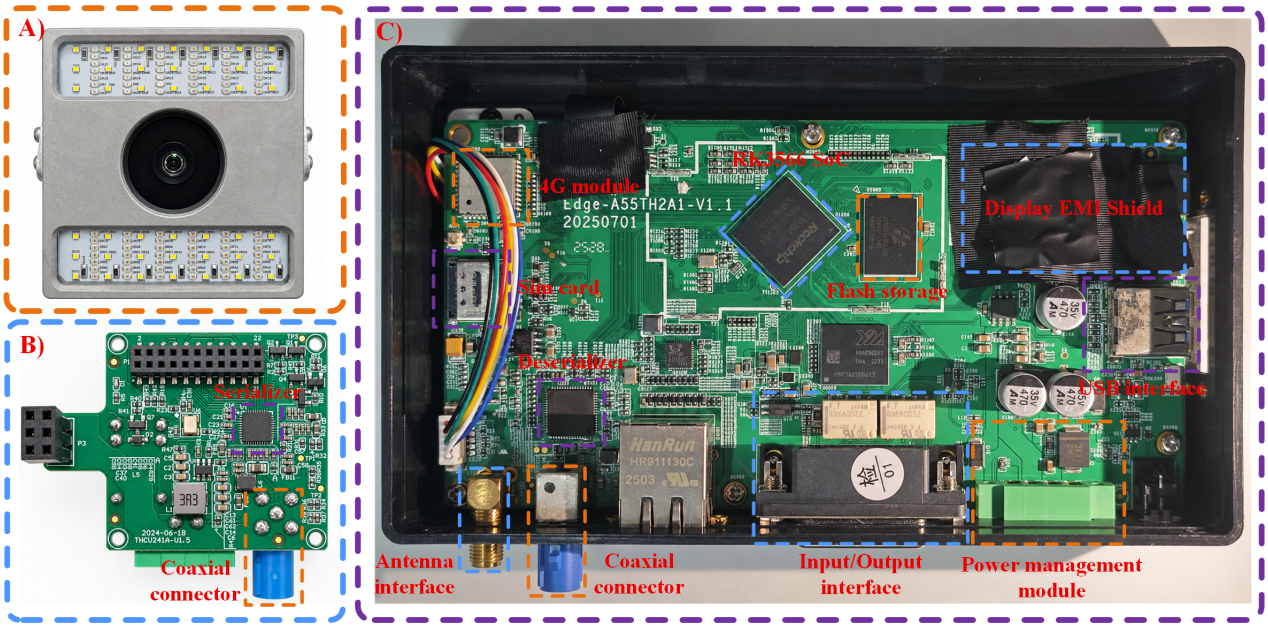


Fig. 2. Hardware of the defect detection system: (A) industrial camera with LED illumination; (B) camera-side interface board with a serializer and coaxial connector; and (C) RK3566 edge board with the 4G communication, flash storage, camera-interface, input/output interface, and power-management components.

include the start button, Hall sensor, industrial camera, and RK3566 edge device.

III. DETECTION ALGORITHM

A. MCGI-YOLO Architecture

Fig. 4 presents MCGI-YOLO, which is built upon YOLOv8n [27]. MCE is embedded in the backbone C2f blocks at P3 (stride 8) and P4 (stride 16). It introduces horizontal and vertical context to model the directional continuity of elongated defects while preserving the original local feature paths.

GIU replaces the two nearest-neighbor upsampling operations in the top-down neck. It adapts the balance between response replication and smoothing to accommodate variations in defect morphology and scale before fusion with lateral backbone features. The remaining backbone stages, bottom-up aggregation path, and three-scale detection head are retained [28], [29].

B. Morphology-Aware Context Encoding

Horizontal and vertical elongated defects exhibit directional continuity that benefits from spatially extended context. MCE explicitly introduces this directional morphology prior through contextual aggregation, as shown in Fig. 5. Its added mechanism targets the directional structure of elongated defects. The original C2f convolutional paths continue to extract local appearance cues, including those used for hole detection. The defect categories describe dominant fabric directions and allow small deviations from the image axes. In the implementation, row and column pooling provides compact spatial descriptors; this operation is related to the directional encoding principle of coordinate attention [30]. The descriptors are combined with depthwise filtering and residual modulation of a partial hidden branch to complement the local feature paths.

1) Partial-Channel Placement: For an input $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$, the first C2f convolution produces two hidden branches with $c = \lfloor C_{\text{out}}/2 \rfloor$ channels each. Omitting the batch dimension, the original feature extraction is

$$(Z_0, Z_1) = \text{Split}\big(\phi_1(X)\big), \tag{3}$$

$$Z_{i+1} = \mathcal{B}_i(Z_i), \quad i = 1, \ldots, n, \tag{4}$$

where ϕ_1 is the original 1×1 convolution with batch normalization and SiLU, and $\mathcal{B}_i$ is an unchanged bottleneck. MCE acts only on the final hidden feature $F = Z_{n+1}$. It is

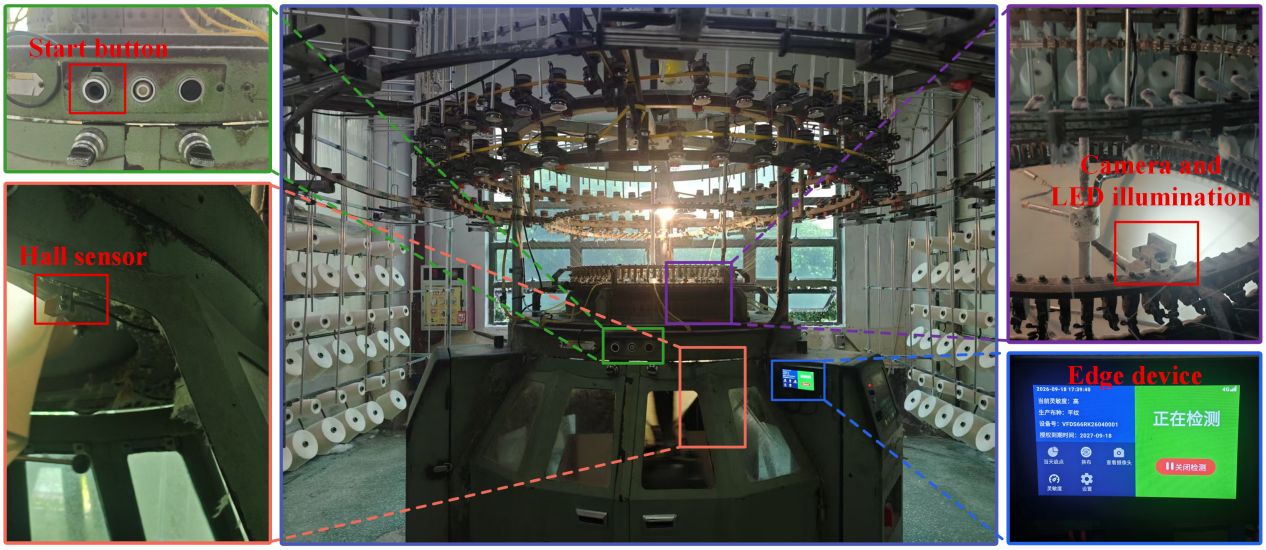


Fig. 3. Actual deployment on the circular knitting machine. The insets highlight the start button, Hall sensor, industrial camera, and RK3566 edge device, respectively.

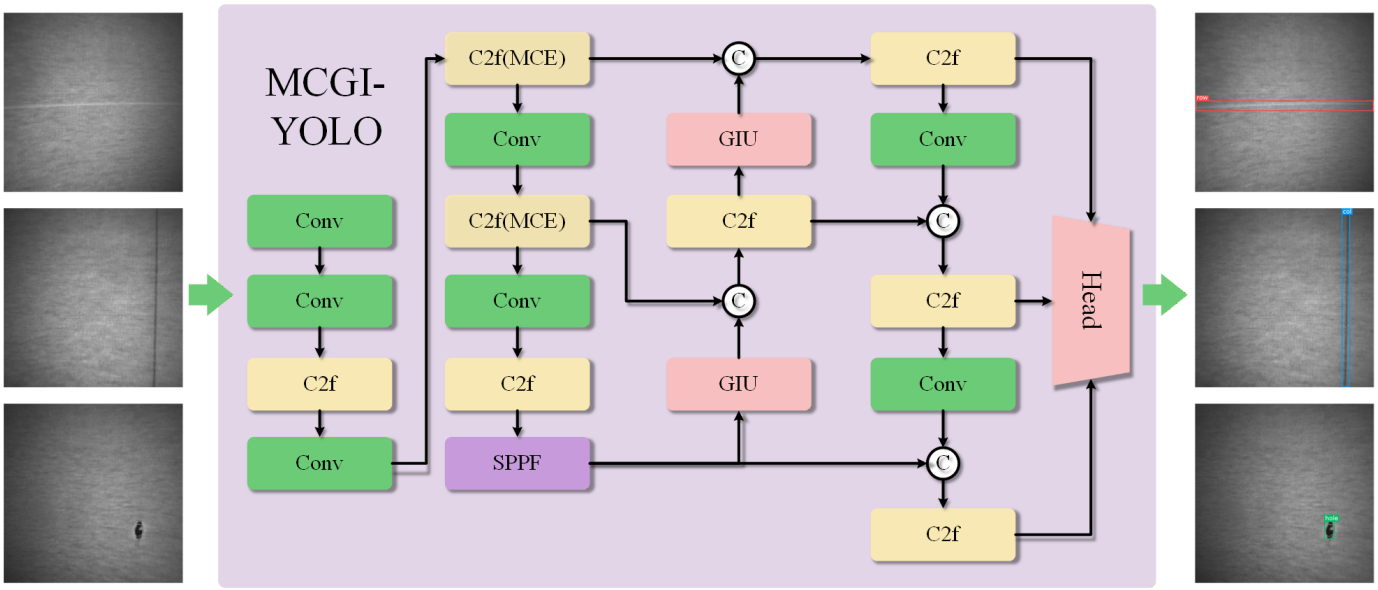


Fig. 4. Architecture of MCGI-YOLO. MCE operates within the stride-8 and stride-16 backbone C2f stages. GIU replaces the two top-down upsampling operations. The local bottlenecks, bottom-up aggregation path, and three-scale detection head are retained from YOLOv8n.

split into an active part $F_a \in \mathbb{R}^{q\times H\times W}$ and a bypass part $F_b \in \mathbb{R}^{(c-q)\times H\times W}$, with

$$q = \max\big(1, \lfloor \rho c \rfloor\big), \qquad \rho = 0.25. \tag{5}$$

Thus, the one-quarter ratio refers to the final hidden branch, rather than the complete C2f input or output.

2) Spatial Context Aggregation: For active channel a, the row and column descriptors are

$$R_h(a,h,1) = \frac{1}{W}\sum_{w=1}^{W} F_a(a,h,w), \tag{6}$$

$$R_w(a,1,w) = \frac{1}{H}\sum_{h=1}^{H} F_a(a,h,w). \tag{7}$$

Here, $R_h \in \mathbb{R}^{q\times H\times 1}$ aggregates the full width while retaining row positions, and $R_w \in \mathbb{R}^{q\times 1\times W}$ aggregates the full height while retaining column positions. The descriptors are filtered along their retained axes and combined as

$$T = \mathrm{DW}_{3\times 1}(R_h) + \mathrm{DW}_{1\times 3}(R_w). \tag{8}$$

The sum uses spatial broadcasting to obtain $T \in \mathbb{R}^{q\times H\times W}$. Both depthwise convolutions have a bias and preserve the descriptor dimensions through padding; no batch normalization or activation is inserted between these filters and the sum. The long-range support comes from the full-axis means, while the small kernels filter the resulting descriptors.

3) Residual Modulation and Feature Reuse: The context

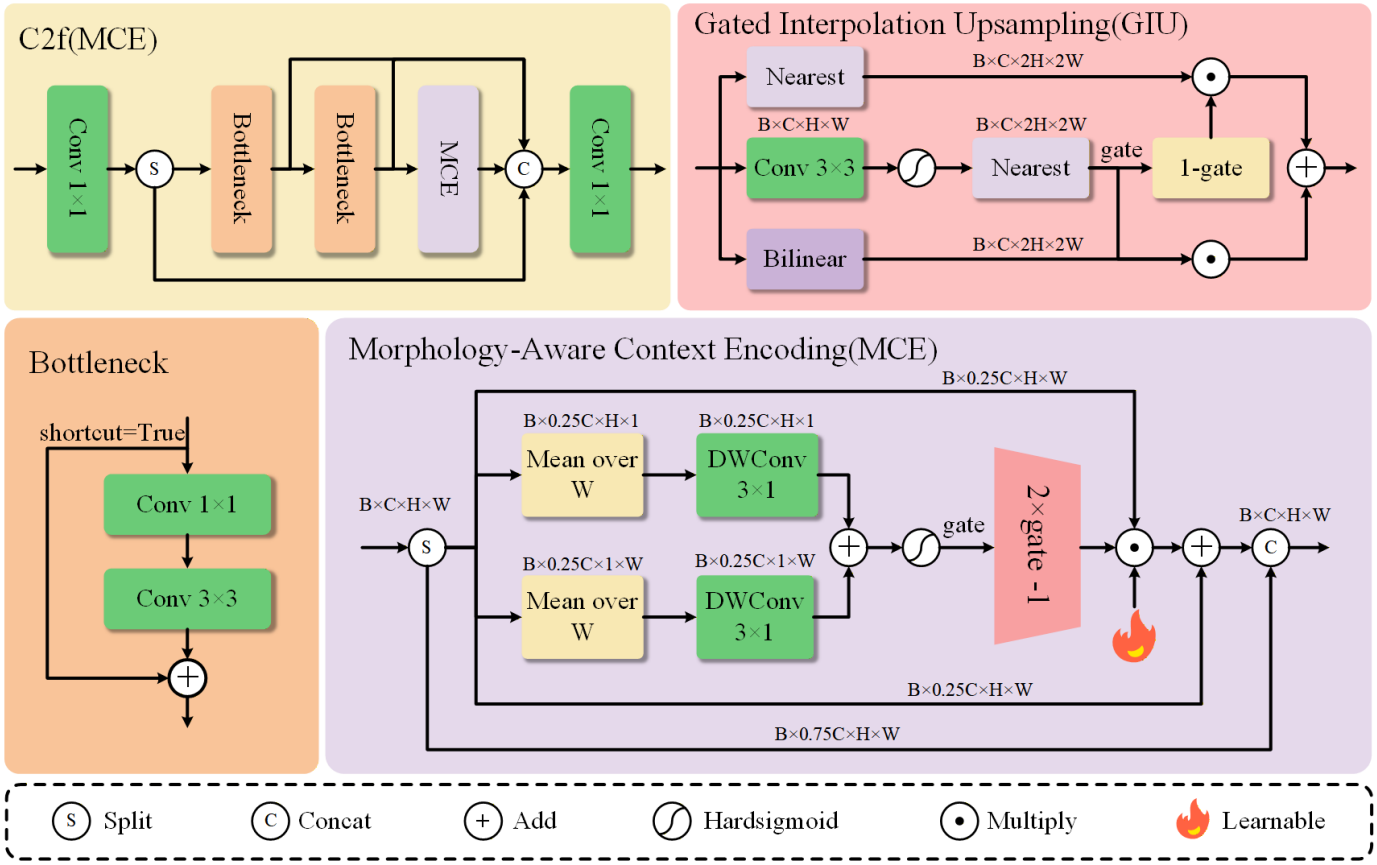


Fig. 5. Structures of C2f with MCE and GIU. MCE embeds directional context modulation in a partial C2f branch while retaining local feature paths. GIU blends nearest-neighbor and bilinear interpolation using a spatial gate shared across channels.

response is converted into a signed modulation map:

$$A = 2\,\mathrm{HS}(T) - 1, \tag{9}$$

$$F'_a = F_a + \gamma \odot F_a \odot A, \tag{10}$$

where $\mathrm{HS}(t) = \min(1, \max(0, (t+3)/6))$ is Hardsigmoid, $\odot$ denotes element-wise multiplication with broadcasting, and $\gamma \in \mathbb{R}^{q\times1\times1}$ is a learnable per-channel scale. Although $A \in [-1, 1]$, no amplitude constraint is imposed on γ. The unchanged bypass channels are concatenated with the modulated channels, and the C2f output is

$$Y = \phi_2\big([Z_0, \ldots, Z_n, [F'_a, F_b]]\big), \tag{11}$$

where ϕ_2 is the original final 1×1 convolution with batch normalization and SiLU. The earlier branches and all original bottlenecks are reused, so directional aggregation complements an existing local feature path rather than replacing it.

The scale γ is initialized to zero. Consequently, a MCE block starts with the same forward mapping as its corresponding C2f block when their shared parameters are identical. During learning, γ controls the introduction of the directional correction. The two depthwise filters and channel scale add $9q$ parameters in total: $6q$ filter weights, $2q$ biases, and q scale parameters. Descriptor filtering costs $3q(H + W)$ multiply–accumulate operations; pooling, broadcasting, and modulation additionally process the $H \times W$ feature grid. Partial-channel placement reduces this overhead while retaining the original local feature extraction paths.

C. Gated Interpolation Upsampling

Circular-knitting defects exhibit variations in morphology and scale, including horizontally and vertically elongated flaws and holes of different sizes. These variations place different demands on response continuity, local concentration, and boundary representation during feature reconstruction. Fixed interpolation applies the same replication or smoothing rule throughout the feature map, motivating a content-adaptive interpolation strategy for heterogeneous local structures.

The lateral stride-8 and stride-16 backbone features, denoted F_3 and F_4, provide local texture and localization cues. GIU adapts the reconstruction of higher-level context before concatenation with these lateral features (Fig. 5). Nearest-neighbor interpolation replicates source responses, whereas bilinear interpolation combines neighboring responses using fixed geometric weights. A compact spatial gate, predicted from the coarse input features, adjusts the balance between these two behaviors according to local feature content. The reconstructed context and the retained lateral features then jointly participate in fusion. Gated combinations of transformed and bypass signals have established precedents in highway networks [31]; the present design applies complementary gating to two fixed interpolation paths.

1) Two Fixed Interpolation Paths: For $X \in \mathbb{R}^{C\times H\times W}$, the

two outputs are

$$N = \mathrm{Nearest}_2(X), \tag{12}$$
$$B = \mathrm{Bilinear}_2(X), \tag{13}$$

where $N, B \in \mathbb{R}^{C\times 2H\times 2W}$. Bilinear interpolation uses `align_corners=False`. Neither path changes the channel dimension or introduces learned interpolation kernels.

2) Shared Spatial Gate: The gate is predicted at the input resolution:

$$G = \mathrm{Nearest}_2\big(\mathrm{HS}(W_g * X + b_g)\big), \tag{14}$$

where $*$ denotes convolution, $W_g \in \mathbb{R}^{1\times C\times 3\times 3}$, and b_g is a scalar bias. Padding of one preserves the low-resolution spatial dimensions, and no batch normalization is used. The gate $G \in [0,1]^{1\times 2H\times 2W}$ is broadcast across channels. Each low-resolution gate value is replicated over a 2×2 output block, avoiding an independent gate predictor at the higher resolution.

3) Gated Reconstruction: The two paths are combined as

$$Y = N + G \odot (B - N) \tag{15}$$
$$= (1 - G) \odot N + G \odot B. \tag{16}$$

For any channel and position, Y lies between the corresponding values of N and B. A gate near zero favors replication, whereas a gate near one favors bilinear interpolation. If the two paths agree, the gate has no effect. This constrains the learned correction to the difference between the two interpolation results; it does not synthesize an unrestricted residual feature. In particular, spatially constant input features remain constant after GIU, including at image boundaries under these interpolation settings.

The gate convolution is initialized with $W_g = 0$ and $b_g = -2.7$, giving

$$G_{\mathrm{init}} = 0.05, \qquad Y_{\mathrm{init}} = 0.95N + 0.05B. \tag{17}$$

The initial interpolation rule is therefore the same for a fixed input across random seeds, while the remainder of the detector still depends on initialization and training. Each GIU module adds $9C + 1$ learned parameters, and its gate convolution costs $9CHW$ multiply–accumulate operations. The two interpolation paths, gate upsampling, and output mixing incur additional operations and memory traffic that must be included in device-level measurements.

MCE and GIU address the encoding and reconstruction stages of morphology-informed feature learning. MCE encodes the horizontal and vertical structure of elongated defects through directional context aggregation while retaining the original local feature paths; GIU adapts the interpolation of higher-level context to local feature content, complementing the texture and localization cues retained in the lateral backbone features. Together, they connect directional context encoding with content-dependent spatial reconstruction within a small parameter budget. The individual and joint effects are evaluated through the four configurations defined in Section V-B.

IV. Experimental Setup

A. Dataset

The dataset was collected from a real circular-knitting production line and covers three defect categories: horizontal defects(*row*), vertical defects(*col*), and holes(*hole*). (*row*). It comprises 403 images with 468 annotated defect instances, divided into training, validation, and test sets of 282, 40, and 81 images, respectively, as summarized in Table II.

TABLE II
IMAGE-LEVEL SPLIT AND DEFECT-INSTANCE COUNTS IN THE REAL PRODUCTION-LINE DATASET.

Split	Images	Instances	Row	Col	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

B. Implementation Details

Table III summarizes the training and deployment settings. MCGI-YOLO and its component-ablation variants share the same `yolov8n.pt` initialization as the YOLOv8n baseline, and all component comparisons use a common training protocol with matched random seeds. Other comparison models use available pretrained checkpoints from their respective implementations.

The RT-DETR-R18 entry uses the RT-DETRv2 implementation from the official repository [32].

C. Evaluation Metrics

Intersection over union (IoU) measures the overlap between predicted and ground-truth bounding boxes as the ratio of their intersection area to their union area. Detection accuracy is evaluated using mAP50 and mAP50–95. Let $a_c(t)$ denote the

TABLE III
IMPLEMENTATION PROTOCOL. TRAINING SETTINGS DESCRIBE THE ULTRALYTICS EXPERIMENTS; DEPLOYMENT SETTINGS DESCRIBE THE INSTALLED EDGE DEVICE.

Category	Parameter	Specification
Training setup	Hardware	Intel i7-13700KF; NVIDIA RTX 3090 (24 GB)
	Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
Deployment setup	Hardware	Rockchip RK3566
	System	Android 13
	Quantization	INT8
Data protocol	Input resolution	Fixed at 640×640
Training dynamics	Batch	16
	Maximum epochs	300
	Patience	80 epochs
	Optimizer	AdamW

TABLE IV
DETECTOR COMPARISON ON THE TEST SET. FPS DENOTES FP32 MODEL-FORWARD THROUGHPUT ON AN RTX 3090 WITH BATCH SIZE 1 AND A 640×640 INPUT. BOLD AND UNDERLINED VALUES INDICATE THE BEST AND SECOND-BEST RESULTS, RESPECTIVELY.

Model	Params (M)	FLOPs (G)	mAP50 (%)	mAP50–95 (%)	FPS
YOLOv9t [33]	**2.12**	8.5	90.1	53.3	291
YOLOv10n [34]	2.78	8.8	88.2	58.5	<u>524</u>
YOLO11n [27]	2.62	<u>6.7</u>	93.3	60.8	463
YOLO12n [35]	2.60	7.6	89.0	56.8	326
YOLO26n [27]	<u>2.57</u>	**6.2**	90.5	58.2	425
RT-DETR-R18 [36]	20.09	60.7	95.2	**67.7**	146
DETR-R50 [37]	41.28	77.3	96.4	53.6	101
Deformable DETR-R50 [38]	39.8	158.7	<u>96.8</u>	64.3	49
Faster R-CNN-R50 [39]	41.1	181.5	96.1	64.7	66
YOLOv8n (Baseline) [27]	3.01	8.1	90.5	58.4	**554**
MCGI-YOLO	3.02	8.2	**97.2**	<u>65.5</u>	486

average precision for class c at IoU threshold t. The metrics are computed as

$$\mathrm{mAP}_{50} = \frac{1}{N_c} \sum_{c=1}^{N_c} a_c(0.50), \tag{18}$$

$$\mathrm{mAP}_{50:95} = \frac{1}{10 N_c} \sum_{c=1}^{N_c} \sum_{t \in T} a_c(t), \tag{19}$$

where $N_c = 3$ and $T = \{0.50, 0.55, \ldots, 0.95\}$. Both metrics are reported as percentages, with improvements expressed in percentage points.

Model parameters and FLOPs are measured at 640×640 input resolution using each model's respective framework. For FPS comparison, all models are evaluated on an RTX 3090 with FP32 precision, batch size 1, and 640×640 inputs. This measurement is separate from the RK3566 deployment result.

V. RESULTS AND DISCUSSION

A. Comparison With Other Detectors

Table IV compares the test-set accuracy, model complexity, and inference throughput of the evaluated detectors. MCGI-YOLO achieves the highest mAP50 of 97.2% and the second-highest mAP50–95 of 65.5%. Compared with the YOLOv8n baseline, these scores increase by 6.7 and 7.1 percentage points, respectively. The parameter count increases from 3.01 M to 3.02 M, and FLOPs increase from 8.1 G to 8.2 G. Model-forward throughput decreases from 554 to 486 FPS on the RTX 3090. Thus, the accuracy improvement comes with only a slight increase in model size and computation, at the cost of a moderate drop in throughput.

Among the other lightweight YOLO variants, YOLO11n achieves the highest accuracy, with 93.3% mAP50 and 60.8% mAP50–95. MCGI-YOLO improves these scores by 3.9 and 4.7 percentage points while achieving 486 FPS compared with 463 FPS for YOLO11n. Several YOLO variants have fewer parameters or lower FLOPs than MCGI-YOLO, but their accuracy is lower on this test set. These results demonstrate that adapting the model to this specific task yields better performance than using generic lightweight detectors.

RT-DETR-R18 obtains the highest mAP50–95 of 67.7%, exceeding MCGI-YOLO by 2.2 percentage points, whereas its mAP50 is 2.0 percentage points lower. This advantage at stricter matching thresholds comes with 20.09 M parameters and 60.7 G FLOPs, compared with 3.02 M and 8.2 G for MCGI-YOLO. Its throughput is also lower, at 146 FPS. MCGI-YOLO additionally outperforms DETR-R50, Deformable DETR-R50, and Faster R-CNN-R50 on both mAP metrics with fewer parameters and FLOPs. Overall, MCGI-YOLO provides a favorable accuracy–efficiency tradeoff among the evaluated configurations, although it does not achieve the highest mAP50–95 or the highest throughput.

TABLE V
COMPONENT ABLATION OF MCE AND GIU.

Variant	MCE	GIU	Params(M)	FLOPs(G)	mAP50	mAP50–95
YOLOv8n	–	–	3.01	8.10	90.5	58.4
+ MCE	✓	–	3.01	8.2	93.9	63.6
+ GIU	–	✓	3.01	8.2	94.4	63.0
MCGI-YOLO	✓	✓	3.02	8.2	97.2	65.5

Fig. 6 compares predictions for horizontal defects, vertical defects, and a hole. In the fourth row, YOLOv8n produces several boxes along a single vertical defect, while MCGI-YOLO produces one box spanning the annotated defect. In the final row, YOLOv8n divides the hole into multiple overlapping predictions, whereas MCGI-YOLO produces a single box that more closely follows the annotated extent. These examples illustrate differences in prediction fragmentation and box localization that complement the aggregate accuracy results.

B. Component Ablation

Table V defines four configurations: the original YOLOv8n baseline, MCE alone, GIU alone, and their combination. The MCE-only model retains nearest-neighbor upsampling, while the GIU-only model retains the original backbone C2f blocks. The comparison isolates the two architectural changes under the common training protocol. The finalized results will report both individual effects and the joint configuration, together with variation across matched seeds and class-specific results for elongated defects and holes.

Feature visualizations in Fig. 7 complement the numerical analysis by comparing responses at corresponding feature stages on the same images. They are interpreted alongside localization

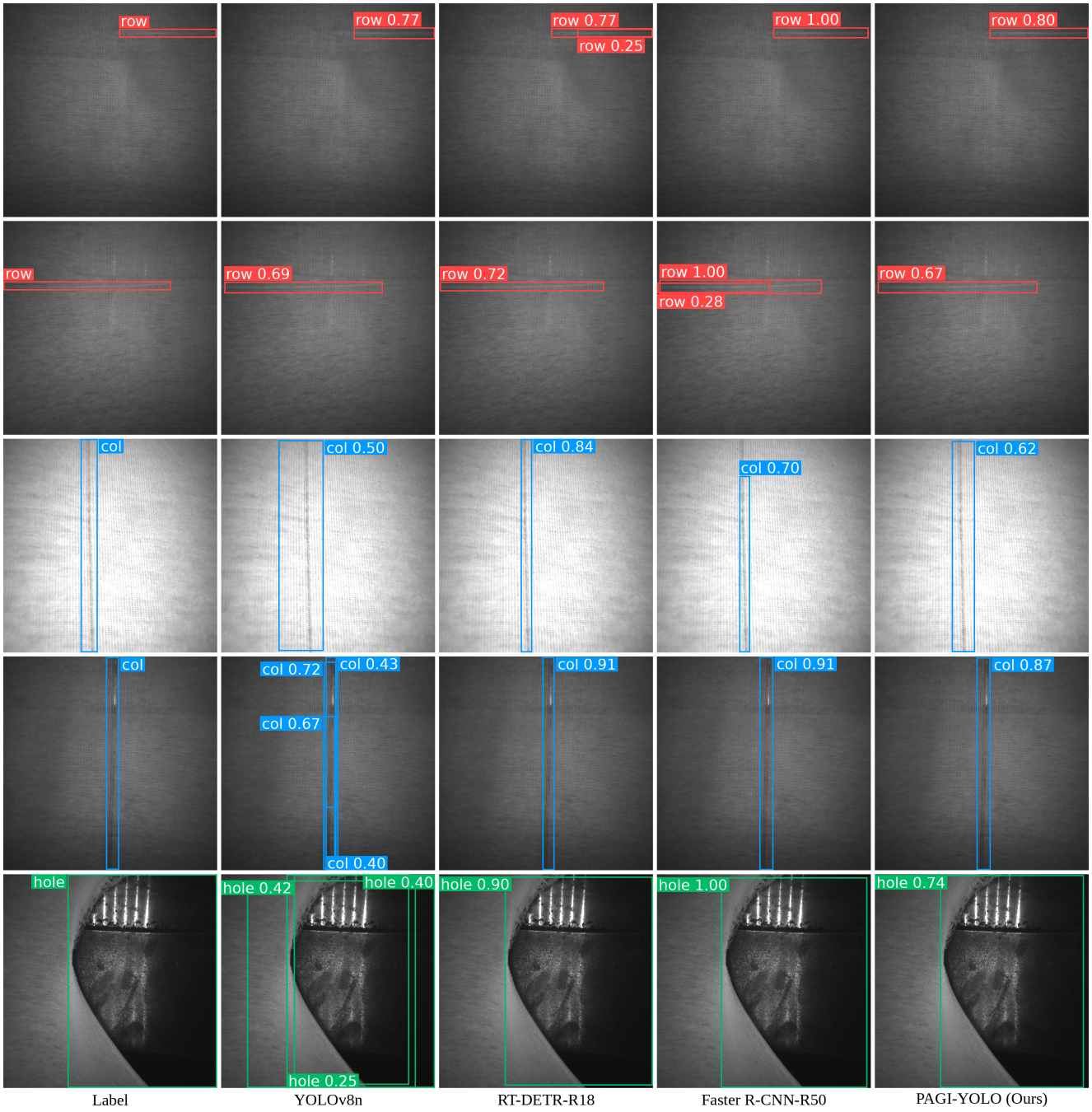


Fig. 6. Qualitative comparison of detection results on selected test images. From left to right: ground-truth annotations (Label), YOLOv8n, RT-DETR-R18, Faster R-CNN-R50, and MCGI-YOLO (Ours).

and classification results rather than as a separate quantitative measure of detection performance.

C. Model Deployment and On-Device Performance

To deploy MCGI-YOLO on the RK3566 edge device, the trained model is exported to ONNX with a fixed input size of 640×640 and then converted into an INT8 RKNN model using RKNN-Toolkit2 [40]. For 640×640 inputs, the deployed INT8 model achieves a measured inference throughput of 22 FPS on the RK3566. This throughput is 2.2 times the 10 FPS design target derived in Section II-D, meeting the model-inference rate required for the specified machine speed and camera field of view.

VI. Conclusion

This paper presents a dedicated defect detection and closed-loop control system for circular knitting machines, together with the lightweight detector MCGI-YOLO. The detector explicitly incorporates directional morphology priors for horizontal and vertical elongated defects. MCE augments retained local C2f processing with extended directional context, while GIU adapts the reconstruction of higher-level context before fusion with

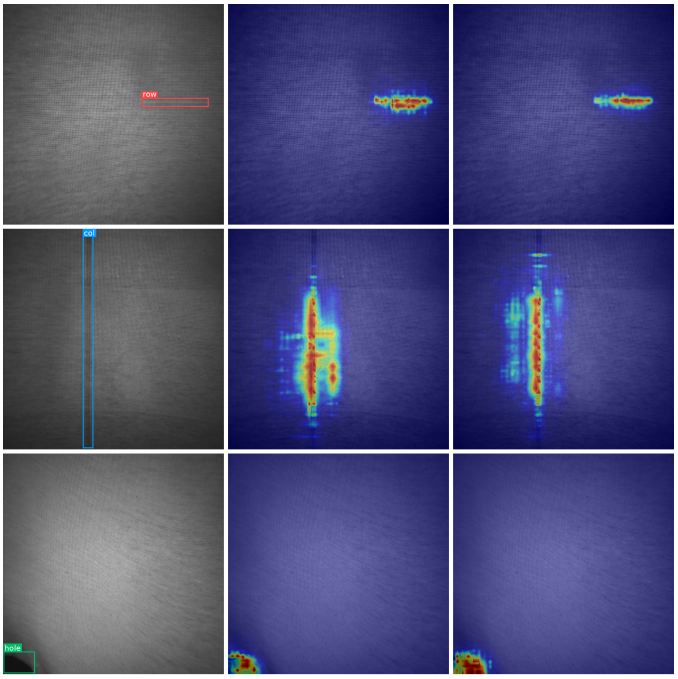


Fig. 7. Feature visualization of YOLOv8n and MCGI-YOLO. From left to right: ground-truth annotations (Label), YOLOv8n, and MCGI-YOLO.

local features to accommodate variations in defect morphology and scale. These complementary designs connect defect morphology modeling with spatial feature reconstruction under the computational constraints of an edge device.

The system integrates image acquisition, SerDes transmission, RK3566 local computation, rotation sensing, and wired machine-stop control. The deployed detector achieves an inference throughput of 22 frames/s on the RK3566. When a defect is detected, local control interrupts production and cloud communication supports defect reporting and worker notification. The component-ablation protocol evaluates MCE, GIU, and their combination against the same baseline; its final numerical findings will be incorporated with the corresponding detector comparison. Quantized-model accuracy, application response time, physical stopping delay, and false-stop frequency define the remaining system-level evaluation, together with generalization across fabric textures and production conditions.

REFERENCES

[1] Cotton Incorporated, "Knit basics," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/knit-basics/. Accessed: Oct. 8, 2026.

[2] The Woolmark Company, "Circular knit technology," [Online]. Available: https://www.woolmark.com/industry/product-development/processing-innovations/circular-knitting-technology/. Accessed: Oct. 8, 2026.

[3] Cotton Incorporated, "Single and double knits," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/single-and-double-knits/. Accessed: Oct. 8, 2026.

[4] International Textile Manufacturers Federation, "Global textile machinery shipments shrunk in 2025, except for spinning," Jul. 2026, [Online]. Available: https://www.itmf.org/images/dl/press-releases/2026/2026-07-24-ITMSS-48-2025.pdf. Accessed: Oct. 10, 2026.

[5] United Nations Environment Programme, "Transforming the textile industry: Learning to manage waste, better," United Nations Environment Programme, Tech. Rep., Oct. 2025, [Online]. Available: https://wedocs.unep.org/handle/20.500.11822/48707. Accessed: Oct. 14, 2026.

[6] S. Takeuchi, K. Nishioka, H. Uematsu, and S. Tanoue, "Research into development of the defect detection system for knitted fabric produced by the circular knitting machines by image analysis," *Journal of Textile Engineering*, vol. 64, DOI 10.4188/jte.64.45, no. 2, pp. 45–49, 2018.

[7] M. M. K. Akter, U. N. Haq, M. M. Islam, and M. A. Uddin, "Textile-apparel manufacturing and material waste management in the circular economy: A conceptual model to achieve sustainable development goal (SDG) 12 for bangladesh," *Cleaner Environmental Systems*, vol. 4, DOI 10.1016/j.cesys.2022.100070, p. 100070, 2022.

[8] Z. Xiang, Y. Shen, M. Ma, and M. Qian, "HookNet: Efficient multiscale context aggregation for high-accuracy detection of fabric defects," *IEEE Trans. Instrum. Meas.*, vol. 72, DOI 10.1109/TIM.2023.3280519, pp. 1–11, 2023, art. no. 5016311.

[9] S. Song, J. Jing, Y. Huang, and M. Shi, "EfficientDet for fabric defect detection based on edge computing," *J. Eng. Fibers Fabrics*, vol. 16, DOI 10.1177/15589250211008346, pp. 1–13, 2021.

[10] R. Machado, L. A. M. Barros, V. Vieira, F. D. da Silva, H. Costa, and V. Carvalho, "Textile defect detection using artificial intelligence and computer vision—a preliminary deep learning approach," *Electronics*, vol. 14, DOI 10.3390/electronics14183692, no. 18, p. 3692, 2025.

[11] T. Yan, Y. Wu, Y. Shi, C. Lin, and L. Ji, "Development and application of an AI visual defect detection system for warp-knitted lace based on 5G+ technology," *Information*, vol. 17, DOI 10.3390/info17070623, no. 7, p. 623, 2026.

[12] Y. Wang, H. Yu, B. Guo, H. Shi, and Z. Yu, "Research on real-time detection system of rail surface defects based on deep learning," *IEEE Sensors J.*, vol. 24, DOI 10.1109/JSEN.2024.3402730, no. 13, pp. 21 157–21 167, 2024.

[13] D. Li, Y. Lu, Q. Gao, X. Li, X. Yu, and Y. Song, "LiteYOLO-ID: A lightweight object detection network for insulator defect detection," *IEEE Trans. Instrum. Meas.*, vol. 73, DOI 10.1109/TIM.2024.3418082, pp. 1–12, 2024, art. no. 5023812.

[14] B. Zhao, Y. Wang, H. Zhang, J. Liu, and J. Zhang, "KAM-Net: Kilobyte-scale ultralightweight attention-based network for glass defect detection with algorithm/hardware co-design," *IEEE Trans. Instrum. Meas.*, vol. 74, DOI 10.1109/TIM.2025.3593602, pp. 1–10, 2025, art. no. 5041310.

[15] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 779–788, 2016.

[16] H. Yang, J. Jing, Z. Wang, Y. Huang, and S. Song, "YOLOV4-TinyS: A new convolutional neural architecture for real-time detection of fabric defects in edge devices," *Textile Research Journal*, vol. 94, DOI 10.1177/00405175231202035, no. 1–2, pp. 49–68, 2024.

[17] X. Kang and J. Li, "AYOLOv7-tiny: Towards efficient defect detection in solid color circular weft fabric," *Textile Research Journal*, vol. 94, DOI 10.1177/00405175231205898, no. 1–2, pp. 225–245, 2024.

[18] C. Chen, Q. Zhou, S. Li, D. Luo, and G. Tan, "Fabric defect detection algorithm based on improved YOLOv8," *Textile Research Journal*, vol. 95, DOI 10.1177/00405175241261092, no. 3–4, pp. 235–251, 2025.

[19] K. Yu, W. Lyu, X. Yu, Q. Guo, W. Xu, and L. Zhang, "FA-YOLO: A high-precision and efficient method for fabric defect detection in textile industry," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E107.A, DOI 10.1587/transfun.2023EAP1030, no. 6, pp. 890–898, 2024.

[20] Z. Zhang, T. Huang, W. Zhang, Y. Yang, and Q. Yu, "DGHR-YOLO: Fabric defect detection based on high-level screening-feature pyramid networks and deformable convolution," *Nondestructive Testing and Evaluation*, vol. 40, DOI 10.1080/10589759.2025.2451768, no. 12, pp. 5919–5944, 2025.

[21] B. Su, H. Chen, and Z. Zhou, "BAF-Detector: An efficient CNN-based detector for photovoltaic cell defect detection," *IEEE Trans. Ind. Electron.*, vol. 69, DOI 10.1109/TIE.2021.3070507, no. 3, pp. 3161–3171, 2022.

[22] X. Shen, J. Liu, L. Jiang, X. Liu, and H. Zhang, "A novel weld defect detection method for intelligent magnetic flux leakage detection system via contextual relation network," *IEEE Trans. Ind. Electron.*, vol. 71, DOI 10.1109/TIE.2023.3294578, no. 6, pp. 6304–6314, 2024.

[23] J. Liu, Z. Wen, X. Shen, F. Zuo, L. Jiang, and H. Zhang, "Online pipeline weld defect detection for magnetic flux leakage inspection system via lightweight rotated network," *IEEE Trans. Ind. Electron.*, vol. 72, DOI 10.1109/TIE.2024.3503635, no. 7, pp. 7573–7584, 2025.

[24] THine Electronics, Inc., "THCV241A: MIPI CSI-2 to V-by-One HS serializer," [Online]. Available: https://www.thine.co.jp/en/products/detail/THCV241A.html. Accessed: Oct. 16, 2026.

[25] THine Electronics, Inc., "THCV242A: V-by-One HS to MIPI CSI-2 deserializer," [Online]. Available: https://www.thine.co.jp/en/products/detail/THCV242A.html. Accessed: Oct. 16, 2026.

[26] Rockchip Electronics, "RK3566 brief datasheet," [Online]. Available: https://www.rock-chips.com/uploads/pdf/2022.8.26/191/RK3566%20Brief%20Datasheet.pdf. Accessed: Oct. 15, 2026.
[27] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.
[28] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, DOI 10.1109/CVPR.2017.106, pp. 2117–2125, 2017.
[29] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, DOI 10.1109/CVPR.2018.00913, pp. 8759–8768, 2018.
[30] Q. Hou, D. Zhou, and J. Feng, "Coordinate attention for efficient mobile network design," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 13 713–13 722, 2021. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021/html/Hou_Coordinate_Attention_for_Efficient_Mobile_Network_Design_CVPR_2021_paper.html
[31] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Training very deep networks," in *Advances in Neural Information Processing Systems*, vol. 28, pp. 2377–2385, 2015. [Online]. Available: https://proceedings.neurips.cc/paper/2015/hash/215a71a12769b056c3c32e7299f1c5ed-Abstract.html
[32] W. Lv *et al.*, "RT-DETR: Official repository with RT-DETRv2 implementations," [Online]. Available: https://github.com/lyuwenyu/RT-DETR, accessed: Oct. 20, 2026.
[33] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. Eur. Conf. Comput. Vis.*, DOI 10.1007/978-3-031-72751-1_1, pp. 1–21, 2025.
[34] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in *Adv. Neural Inf. Process. Syst.*, vol. 37, DOI 10.52202/079017-3429, pp. 107 984–108 011, 2024.
[35] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," in *Adv. Neural Inf. Process. Syst.*, vol. 38, DOI 10.52202/085713-2627, 2025.
[36] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, DOI 10.1109/CVPR52733.2024.01605, pp. 16 965–16 974, 2024.
[37] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, pp. 213–229, 2020.
[38] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.
[39] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.
[40] Rockchip, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.

Kangzhao Tan received the B.S. degree from Guangzhou University, Guangzhou, China, in 2022. From 2022 to 2024, he worked as a cloud computing R&D engineer with Sangfor Technologies Inc., Shenzhen, China. He is currently pursuing the M.S. degree in computer technology with the School of Computer Science, Guangdong University of Technology, Guangzhou, China. His research interests include deep learning, computer vision, and industrial defect detection.

Jiarui Zhang is currently a graduate student with the School of Computer Science, Guangdong University of Technology, Guangzhou, China. His research interests include hardware circuit design and IoT system.

Junjie Yu is currently a graduate student with the School of Computer Science, Guangdong University of Technology, Guangzhou, China. His research interests include image processing and edge artificial intelligence.

Heng Wu received the B.S. degree in electronic and information engineering from Wuhan University of Science and Technology, Wuhan, China, in 2009, and the M.S. degree in optics and the Ph.D. degree in mechanical manufacturing and automation from South China University of Technology, Guangzhou, China, in 2012 and 2017, respectively.

He is currently an associate professor with the School of Automation, Guangdong University of Technology. His research interests include optical imaging, optical measurement, machine vision, and image processing.

Jing Lin received the B.S. degree in computer science and technology and the M.S. degree in software engineering from Guangdong University of Technology, Guangzhou, China, in 2003 and 2006, respectively.

Since graduation, he has served as an assistant engineer, engineer, and senior engineer. He is currently a senior engineer with the Classroom Management Center, Guangdong University of Technology. His current research interests include information system project management, system informatization, and the intelligent Internet of Things.

Chunhua He received the B.S. degree in microelectronics from Sun Yat-sen University, Guangzhou, China, in 2010, and the M.S. and Ph.D. degrees in microelectronics from Peking University, Beijing, China, in 2013 and 2018, respectively.

From 2013 to 2017, he was an engineer with the No. 5 Electronics Research Institute, Ministry of Industry and Information Technology, Guangzhou, China. From 2017 to 2019, he was an engineer with Midea Group, Foshan, China. From 2019 to 2021, he was a senior engineer with Guangzhou 37 Degree Smart Home Co., Ltd., Guangzhou, China. He joined the School of Computer Science, Guangdong University of Technology, Guangzhou, China, in 2021, where he is currently an professor. His current research interests include the design and application of MEMS sensors and artificial intelligence algorithms.